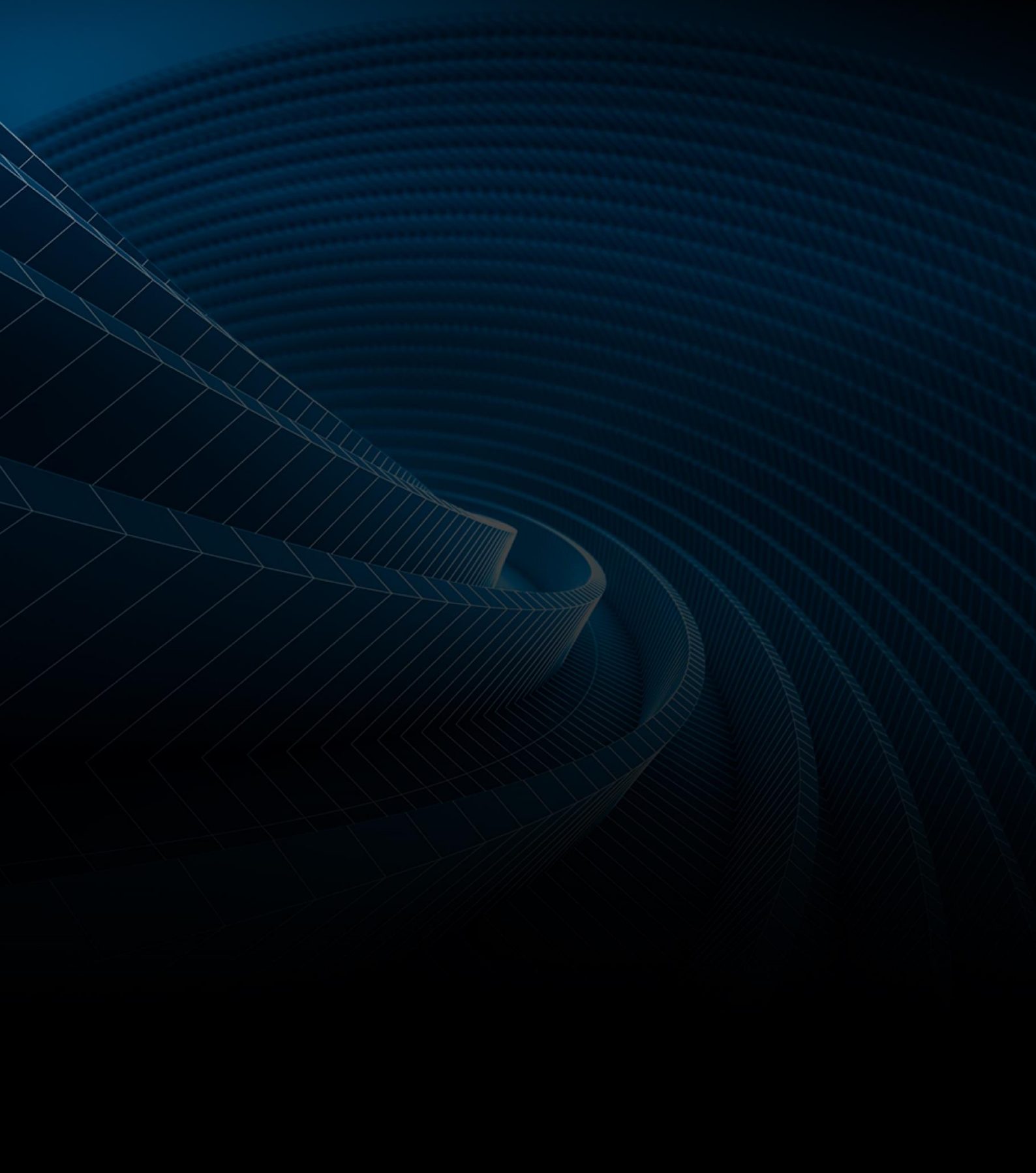


POSTECH



## TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

### O PROBLEMA

A FIAP Cloud Games (FCG) agora opera com uma arquitetura de microsserviços, o que trouxe agilidade e escalabilidade. No entanto, novos desafios surgiram:

- **Exposição de Serviços:** expor cada microsserviço diretamente para a internet é inseguro e complexo para os clientes, que precisam saber o endereço de cada serviço.
- **Visibilidade do Sistema:** quando um erro ocorre, é quase impossível rastrear qual serviço falhou. Não temos visão do desempenho, da latência ou da saúde geral da aplicação. Estamos "voando às cegas".
- **Otimização de Recursos:** o serviço de notificações (NotificationsAPI) passa a maior parte do tempo ocioso, aguardando eventos. Manter um container rodando 24/7 para uma tarefa tão esporádica está se provando um desperdício de recursos computacionais.
- **Performance e Latência:** com o aumento do tráfego, a dependência exclusiva de bancos relacionais e buscas simples tornou-se um impeditivo. É necessário otimizar a persistência de dados e a velocidade de resposta aos usuários.

A FCG precisa de um ponto de entrada unificado, uma forma de otimizar o custo de componentes ociosos e, acima de tudo, de visibilidade sobre o comportamento de seu sistema distribuído.

## O Desafio

O objetivo desta fase é profissionalizar a arquitetura de microsserviços, aplicando as ferramentas de mercado vistas nas disciplinas. Vocês irão:

- Implementar um API Gateway: utilizar Kong, Azure APIM ou AWS API Gateway como ponto de entrada único para gerenciar e proteger o tráfego.
- Migrar para Serverless: otimizar recursos migrando a NotificationsAPI para uma arquitetura de Funções como Serviço.
- Implementar Observabilidade: escolher entre uma stack de código aberto com Prometheus e Grafana ou uma plataforma de APM como Datadog ou New Relic para obter insights profundos da aplicação.
- Persistência Poliglota: implementar MongoDB ou DynamoDB para lidar com dados não estruturados ou de alta escala.
- Otimização com Cache: implementar uma camada de cache (ex.: Redis) para reduzir latência e carga nos bancos de dados.

## Funcionalidades Obrigatórias

### 1. Implementação de um API Gateway

- Introduza um API Gateway como a única porta de entrada para o sistema FCG. As ferramentas recomendadas, conforme a disciplina, são:
  - **Kong API Gateway:** ideal para ambientes Kubernetes, com implantação via manifestos (.yaml).
  - **(Opcional) Azure API Management (APIM) ou AWS API Gateway:** para quem desejar explorar a integração com serviços de nuvem gerenciados.

O Gateway será responsável por:

- Receber todas as requisições externas.
- Validar token JWT.
- Roteamento de requisições para os serviços UsersAPI e CatalogAPI.

## 2. Migração para Arquitetura Serverless

- Refatore o microserviço NotificationsAPI para uma Função Serverless (AWS Lambda, Azure Function ou Google Cloud Function).
- A função deve ser configurada para ser acionada (triggered) diretamente por novas mensagens na fila/tópico do seu sistema de mensageria, substituindo o container que rodava continuamente.

## 3. Implementação da Stack de Observabilidade (Escolha uma das opções)

- Opção A: Stack de Código Aberto
  - Métricas com Prometheus e Grafana:
    - Instrumente os microserviços UsersAPI e CatalogAPI para expor métricas no formato Prometheus.
    - Crie um dashboard no Grafana para visualizar em tempo real: latência de requisições, contagem de requisições (total e por status code HTTP) e taxa de erros.
  - (O Zabbix é excelente para monitoramento de infraestrutura, mas para esta parte do desafio focada em aplicação, a stack Prometheus/Grafana é mais indicada).
- Opção B: Plataforma de APM Gerenciada
  - Ferramentas: Datadog ou New Relic.
  - Configure os agentes de APM nos seus microserviços (UsersAPI, CatalogAPI, PaymentsAPI) e na sua Função Serverless.
  - Utilize a plataforma escolhida para atingir todos os três pilares da observabilidade:
    - Métricas: crie um dashboard para visualizar as mesmas métricas da Opção A (latência, throughput, erros).
    - Logs: envie todos os logs da aplicação para a plataforma e utilize sua ferramenta de busca para análise.
    - Traces: capture e analise o trace distribuído do fluxo de "Compra de Jogo".

#### 4. Persistência Poliglota e Alta Performance

- A arquitetura deve evoluir para suportar diferentes tipos de dados e cargas de trabalho, garantindo baixa latência.
- NoSQL (Obrigatório): Implementar MongoDB ou DynamoDB.
  - Cenário de uso: deve ser utilizado para dados flexíveis ou de alta volumetria (ex.: catálogo expandido, logs de eventos, perfis de usuário ou sistema de avaliações).
  - Implementação: utilizar os drivers oficiais para .NET (MongoDB.Driver ou AWSSDK.DynamoDBv2).
- Cache de Persistência (Obrigatório): implementar uma camada de cache distribuído (ex.: Redis).
  - Cenário de uso: persistir temporariamente dados de sessões, resultados de consultas onerosas ou configurações globais para reduzir o round-trip ao banco de dados principal.
  - Implementação: utilizar a biblioteca StackExchange.Redis ou abstrações do ASP.NET Core (IDistributedCache).

#### Requisitos Técnicos

- **API Gateway:** a configuração do gateway (rotas, políticas) deve ser versionada no repositório de orquestração.
- **Serverless:** o código da função e sua configuração de infraestrutura como código (ex.: SAM, Serverless Framework, Terraform, ou equivalente) devem estar em seu próprio repositório.
- **Observabilidade:** a escolha da stack (Opção A ou B) deve ser documentada no README.md do repositório de orquestração. Para a Opção A, a implantação das ferramentas deve ser feita via manifestos Kubernetes. Para a Opção B, as chaves de API devem ser gerenciadas via Kubernetes Secrets.

- NoSQL: implementar **MongoDB** ou **DynamoDB** e implementar uma camada de cache distribuído (ex.: Redis).

### Entregáveis da Fase 3

- **Vídeo de até 20 minutos** demonstrando a arquitetura em funcionamento:
  - Realizar requisições via Gateway, mostrando o roteamento e a segurança.
  - Demonstrar a Função Serverless sendo acionada e exibir seus logs na plataforma centralizada.
  - **Apresentar a solução de observabilidade escolhida:**
    - **Se Opção A:** mostrar o dashboard do Grafana com métricas em tempo real.
    - **Se Opção B:** mMostrar o dashboard equivalente e o trace distribuído dentro da plataforma Datadog ou New Relic.
  - Explicar como o NoSQL foi integrado à arquitetura
- **Códigos-fonte nos repositórios:**
  - Atualizar os repositórios dos microsserviços com a instrumentação de observabilidade.
  - Link para o novo repositório da Função Serverless.
  - Atualizar o repositório de orquestração com os manifestos do API Gateway e da stack de monitoramento (se Opção A).
  - Código atualizado com os novos drivers de NoSQL e Cache.
  - O README.md do repositório de orquestração deve ser o guia central, explicando a stack escolhida e como subir todo o ambiente.
- **Relatório de entrega (PDF ou TXT) – esse arquivo deve ser postado na data da entrega, contendo:**
  - Nome do grupo.
  - Participantes e usernames no Discord.

- Link da documentação.
- Link do(s) repositório(s).
- Link do vídeo salvo no Youtube ou lugar de sua preferência.

Lembramos que, caso você tenha qualquer dúvida, é só nos chamar no Discord!

